Data split và cross-validation

Chia dữ liệu để điểm validation đáng tin.

> Câu lạc bộ lưu ba bài làm của mỗi học sinh. Ta muốn đoán học sinh ở **một trường mới** có cần gợi ý khi giải bài hay không. Nếu chia ngẫu nhiên từng bài làm, một học sinh có thể xuất hiện ở cả train và validation. Mô hình đã thấy cách trình bày của học sinh ấy; điểm validation có còn đo được khả năng với người mới không?

Không có một cách chia đúng cho mọi bài toán. Cách chia dữ liệu (data split) cần mô phỏng người hoặc thời điểm sẽ xuất hiện lúc sử dụng. Cross-validation lặp lại phép đánh giá trên nhiều phần giữ riêng, nhưng lặp một split sai nhiều lần vẫn trả lời sai câu hỏi.

SAU BÀI NÀY, CÓ THỂ

- Chọn split ngẫu nhiên, stratified, group hoặc theo thời gian từ mục tiêu sử dụng.
- Kiểm tra không trùng hàng, không trùng group và không học từ tương lai.
- Đọc một vòng K-fold và tạo dự đoán out-of-fold đúng vị trí.
- Phân biệt điểm dùng để chọn phương án với test độc lập sau lựa chọn.

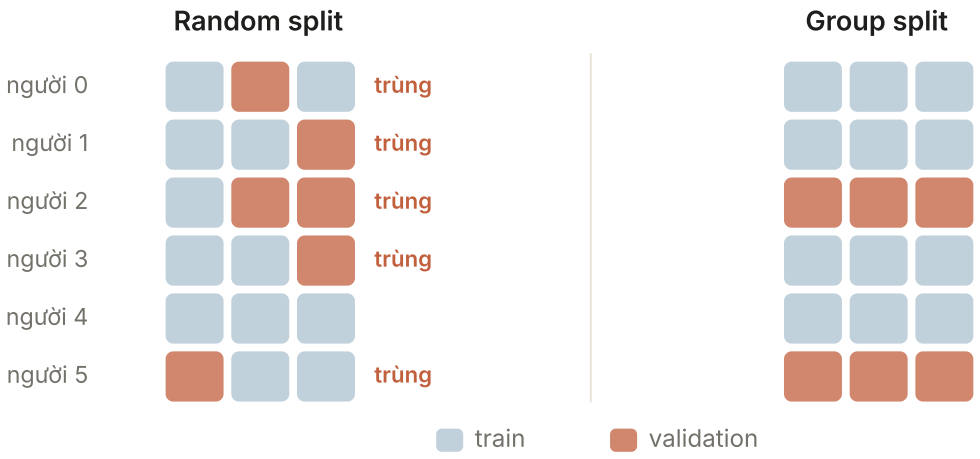
Trước hết, hỏi mẫu mới là ai

Ta cần phân biệt ít nhất ba tình huống. Đoán một bài làm mới của học sinh đã thấy khác với đoán một học sinh chưa thấy. Dự báo buổi sinh hoạt tuần sau khác với đoán một buổi ngẫu nhiên từ cùng giai đoạn.

Tình huống cần mô phỏng	Kiểu split thường phù hợp	Điều cần kiểm
Mẫu độc lập, cùng phân phối	Random holdout hoặc K-fold	Không trùng bản ghi
Classification ít mẫu một lớp	Stratified split/K-fold	Số mẫu từng lớp đủ

Tình huống cần mô phỏng	Kiểu split thường phù hợp	Điều cần kiểm
Người, trường hoặc thiết bị chưa thấy	Group split	Group không xuất hiện ở hai phía
Dự đoán thời điểm tương lai	Split theo thời gian	Train đứng trước validation

Bảng chỉ là điểm xuất phát; chọn split là đặt một giả định về cách dữ liệu mới xuất hiện. Nếu vừa có thời gian vừa có người lặp lại, phép chia phải thỏa **cả hai yêu cầu của nhiệm vụ**. Ví dụ dự đoán tháng tới ở trường mới phải giữ trường mới ra ngoài train, đồng thời kiểm thời điểm thông tin của những trường train.



Chia ngẫu nhiên: năm người có mặt ở cả hai phía. Group split: mỗi người một phía.

HỎI NHANH

Mô hình sẽ dùng để đoán cho những trường chưa có trong train. Mỗi học sinh có nhiều bản ghi. Ta nên để riêng dữ liệu validation theo đơn vị nào?

A Theo trường

B Theo học sinh, các trường vẫn trộn ở cả hai phía

C Theo từng bản ghi, có giữ tỉ lệ nhãn

D Theo thời gian, để riêng tháng cuối của mọi trường

Stratified split giữ lớp, không giữ người

Classification có nhãn 1 là "cần gợi ý", nhãn 0 là "chưa cần". Nếu lớp 1 hiếm, chia ngẫu nhiên có thể tạo validation không có mẫu lớp 1. Khi ấy không đo được khả năng phát hiện lớp này, và một số metric còn không xác định.

Stratified split cố gắng giữ tỉ lệ lớp ở các phần. Tỉ lệ không nhất thiết khớp tuyệt đối vì số mẫu là số nguyên. Nó không ngăn cùng người nằm ở hai phía; với group và mất cân bằng lớp, cân nhắc `StratifiedGroupKFold` nếu phù hợp.

Python

```python
import numpy as np
from sklearn.model_selection import train_test_split

labels = np.array([1, 1, 1] + [0] * 9)
row_ids = np.arange(len(labels))
train_ids, valid_ids = train_test_split(
    row_ids, test_size=0.25, random_state=13, stratify=labels
)
print("Số mẫu train/valid:", len(train_ids), len(valid_ids))
print("Số nhãn 1 train/valid:", labels[train_ids].sum(), labels[valid_ids].sum())
print("Trùng hàng:", len(set(train_ids) & set(valid_ids)))
```

KẾT QUẢ

```
Số mẫu train/valid: 9 3
Số nhãn 1 train/valid: 2 1
Trùng hàng: 0
```

Validation có ba mẫu nên không thể có đúng 25% mẫu lớp 1. Code giữ một mẫu lớp 1 ở validation, hai ở train. Khi dùng nhiều fold, lớp ít nhất phải có đủ mẫu cho số fold nếu muốn mỗi fold chứa lớp ấy. Không tăng số fold tùy ý để làm điểm trông ổn định.

HỎI NHANH

Mỗi học sinh có nhiều bài làm trong dữ liệu. Ta dùng stratified split. Cách chia này bảo đảm được điều gì?

A Không học sinh nào có bài ở cả hai phía

B Bài trong validation đều làm muộn hơn bài trong train

C Tỉ lệ nhãn ở hai phía gần nhau, và chỉ có vậy

D Mỗi học sinh có đúng một bài trong validation

Group split: kiểm giao của tập người

Code dưới tạo sáu học sinh, mỗi người ba bản ghi. `groups` đi cùng hàng để chỉ ra người sở hữu bản ghi; không cần đưa mã người vào feature. Trước tiên nhìn một random split, rồi kiểm ba vòng `GroupKFold`.

Python

```python
from sklearn.model_selection import GroupKFold

groups = np.repeat(np.arange(6), 3)
X_group = np.zeros((18, 1))                # chỉ dùng để minh họa chỉ số
y_group = np.repeat([0, 1, 0, 1, 0, 1], 3)
all_ids = np.arange(18)
train_ids, valid_ids = train_test_split(
    all_ids, test_size=1/3, random_state=13
)
shared = set(groups[train_ids]) & set(groups[valid_ids])
print("Random split, người trùng:", sorted(map(int, shared)))

group_cv = GroupKFold(n_splits=3)
for fold, (train_ids, valid_ids) in enumerate(
    group_cv.split(X_group, y_group, groups), start=1
):
    shared = set(groups[train_ids]) & set(groups[valid_ids])
    assert not shared
```

```
    print(f"Fold {fold}: valid groups={sorted(map(int, set(groups[valid_ids])))}, trùng=
{len(shared)}")
```

KẾT QUẢ

```
Random split, người trùng: [0, 1, 2, 3, 5]
Fold 1: valid groups=[2, 5], trùng=0
Fold 2: valid groups=[1, 4], trùng=0
Fold 3: valid groups=[0, 3], trùng=0
```

Điều đáng nhìn là giao của hai tập group bằng 0 ở mỗi vòng. Không chỉ kiểm giao của **chỉ số hàng**: hai hàng khác nhau vẫn có thể cùng người. Trong dữ liệu ảnh, cùng ảnh chụp được resize, crop hoặc lưu hai tên file cũng có thể cần gom vào một group để tránh đánh giá trên bản gần trùng.

GroupKFold có thể tạo fold với tỉ lệ nhãn khác nhau, nhất là ít group hoặc group rất lớn. Hãy in số mẫu và số nhãn theo fold. Nếu không có cách chia hoàn hảo, ghi giới hạn ấy thay vì âm thầm thay mục tiêu đánh giá.

HỎI NHANH

Bộ ảnh bài làm viết tay có nhiều ảnh là bản cắt, bản xoay nhẹ, hoặc bản chụp lại từ cùng một tờ giấy. Chia ngẫu nhiên theo ảnh thì validation đúng 99%, nhưng gặp bài của lớp khác chỉ đúng 80%. Ta nên sửa cách chia thế nào?

A Chia sao cho tỉ lệ nhãn ở hai phía bằng nhau

B Gom các ảnh cùng một tờ giấy thành một nhóm, rồi chia theo nhóm

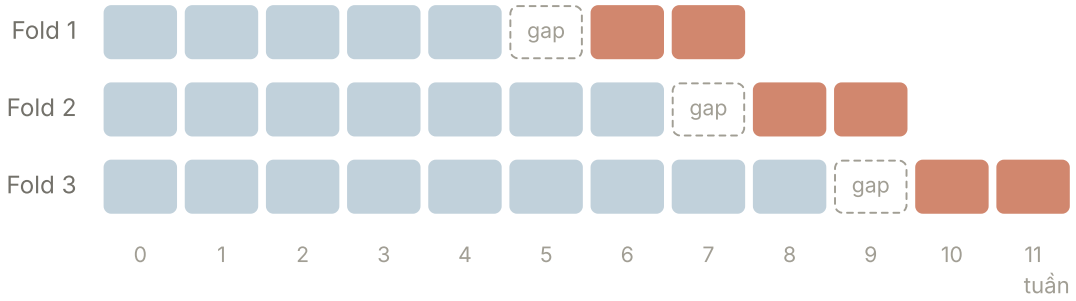
C Chia lại ngẫu nhiên với seed khác cho điểm ổn định hơn

D Bỏ các ảnh cắt khỏi validation nhưng giữ trong train

Thời gian: quá khứ học, tương lai chấm

Muốn dự báo buổi tuần sau, train phải dùng những nhãn đã biết trước lúc dự báo. Xáo trộn có thể cho train chứa tháng sau validation. Mô hình khi ấy được học từ một tương

lai mà lúc sử dụng chưa tồn tại.



Train luôn đứng trước; bỏ một tuần sát ranh giới rồi chấm hai tuần sau.

`TimeSeriesSplit` làm train mở rộng theo thời gian. Code minh họa 12 tuần đã sắp thứ tự, mỗi validation hai tuần, `gap=1` bỏ một tuần sát ranh giới. Trong bài thật, gap phải xuất phát từ thời gian biết nhãn hoặc cửa sổ mục tiêu; số 1 ở đây chỉ là cấu hình để nhìn cơ chế.

Python

```python
from sklearn.model_selection import TimeSeriesSplit

weeks = np.arange(12)
time_cv = TimeSeriesSplit(n_splits=3, test_size=2, gap=1)
for fold, (train_ids, valid_ids) in enumerate(time_cv.split(weeks), start=1):
    assert weeks[train_ids].max() < weeks[valid_ids].min()
    print(f"Fold {fold}: train={train_ids.tolist()}, valid={valid_ids.tolist()}")
```

KẾT QUẢ

```
Fold 1: train=[0, 1, 2, 3, 4], valid=[6, 7]
Fold 2: train=[0, 1, 2, 3, 4, 5, 6], valid=[8, 9]
Fold 3: train=[0, 1, 2, 3, 4, 5, 6, 7, 8], valid=[10, 11]
```

Một feature lag như số đăng ký tuần trước có thể dùng nếu tuần trước đã quan sát được. Không phải cứ hai hàng dùng chung lịch sử là leakage. Vấn đề là liệu feature hoặc nhãn trong train có chứa thông tin chưa biết ở thời điểm validation cần dự đoán.

Nếu các hàng có thời gian không đều, `TimeSeriesSplit` vẫn chia theo số hàng; validation có thể phủ các khoảng thời gian dài ngắn khác nhau. Muốn so cùng một chân trời dự báo, cần chia theo mốc ngày phù hợp. Cũng cần sắp thời gian trước: splitter không tự đoán cột nào là ngày.

HỎI NHANH

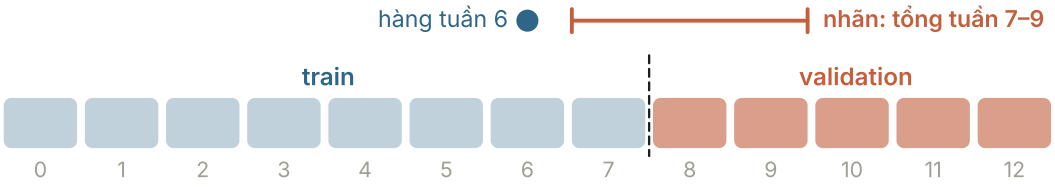
Nhãn của mỗi hàng là tổng số người đến trong ba tuần tiếp theo. Ta chỉ bảo đảm ngày bắt đầu của mọi hàng train sớm hơn validation. Như vậy đã đủ chưa?

A Đủ, vì ngày bắt đầu đã đúng thứ tự

B Đủ, nếu giữ thêm tỉ lệ nhãn

C Chưa, phải bỏ hẳn cột ngày khỏi feature

D Chưa, vì nhãn của train có thể tính cả những tuần thuộc validation



Nhãn là tổng ba tuần sau, nên hàng train sát ranh giới đã nhìn vào validation.

K-fold và dự đoán out-of-fold

Với các mẫu độc lập, K-fold chia dữ liệu thành K phần. Mỗi vòng học trên $K-1$ phần rồi chấm phần còn lại. Một mẫu được chấm ở vòng mà nó không nằm trong train. Tổng điểm thường là trung bình các điểm fold:

$$\bar{s} = \frac{1}{K}\sum_{k=1}^{K} s_k.$$

Khi kích thước fold khác nhau, trung bình không trọng số có thể khác điểm gộp toàn bộ dự đoán. Với metric phi tuyến như F1, ngay cả fold bằng nhau thì trung bình F1 cũng có thể khác F1 của dữ liệu gộp. Ghi rõ đang báo cáo cách nào.

Ta tạo classification giả với 96 mẫu, bốn feature và dùng ba fold stratified. Mô hình `LogisticRegression` trả xác suất lớp 1; phần thuật toán ở bài 2.3. Tại đây mục tiêu là

kiểm chỉ số và tạo dự đoán out-of-fold (OOF), tức dự đoán mỗi mẫu từ một mô hình chưa học mẫu đó.

Python

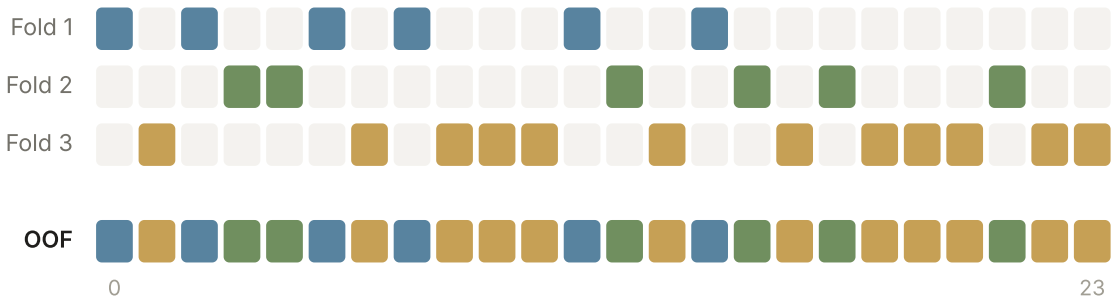
```python
from sklearn.datasets import make_classification
from sklearn.model_selection import StratifiedKFold
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

X, y = make_classification(
    n_samples=96, n_features=4, n_informative=3,
    n_redundant=0, random_state=13,
)
cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=13)
oof_probability = np.full(len(y), np.nan)
times_scored = np.zeros(len(y), dtype=int)
fold_scores = []
for fold, (train_ids, valid_ids) in enumerate(cv.split(X, y), start=1):
    model = LogisticRegression(max_iter=500).fit(X[train_ids], y[train_ids])
    probability = model.predict_proba(X[valid_ids])[:, 1]
    oof_probability[valid_ids] = probability       # ghi về đúng vị trí gốc
    times_scored[valid_ids] += 1
    score = accuracy_score(y[valid_ids], probability >= 0.5)
    fold_scores.append(score)
    print(f"Fold {fold}: n={len(valid_ids)}, accuracy={score:.3f}")
assert np.all(times_scored == 1)
assert np.isfinite(oof_probability).all()
print("OOF shape:", oof_probability.shape)
print(f"Mean fold accuracy={np.mean(fold_scores):.3f}")
print(f"Pooled OOF accuracy={accuracy_score(y, oof_probability >= 0.5):.3f}")
```

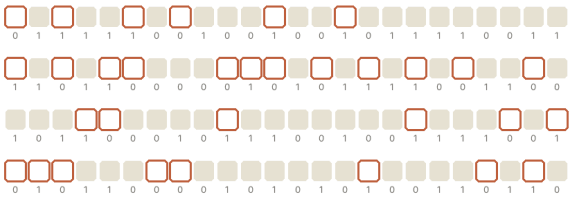
KẾT QUẢ

```
Fold 1: n=32, accuracy=0.969
Fold 2: n=32, accuracy=0.906
Fold 3: n=32, accuracy=0.906
OOF shape: (96,)
Mean fold accuracy=0.927
Pooled OOF accuracy=0.927
```

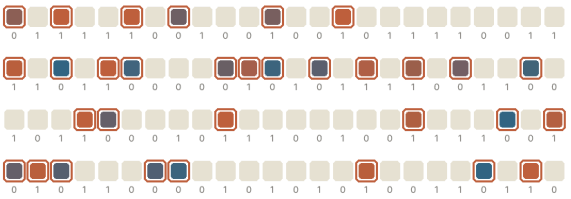
`[:,1]` chọn cột xác suất nhãn 1 trong ví dụ có `model.classes_=[0,1]`. Quan trọng hơn, code ghi kết quả vào `valid_ids`, không nối từng đoạn theo thứ tự fold rồi coi đó là thứ tự gốc. `times_scored` phát hiện một hàng bị thiếu hoặc chấm hai lần.



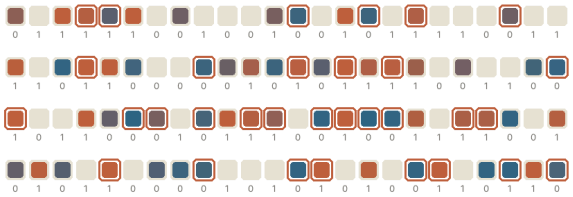
Mỗi fold chấm một phần; OOF ghép về đúng vị trí, mỗi mẫu một lần.



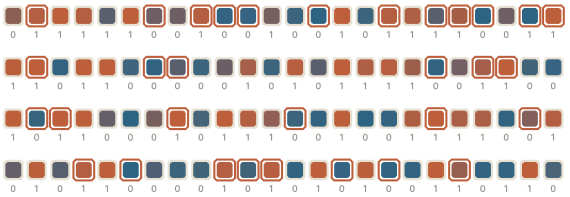
fold 1: train trên 64 mẫu, giữ 32 mẫu viền cam



fold 1: dự đoán ghi về đúng vị trí gốc, accuracy fold 0.969



fold 2: dự đoán ghi về đúng vị trí gốc, accuracy fold 0.906



fold 3: dự đoán ghi về đúng vị trí gốc, accuracy fold 0.906; OOF đủ 96 ô, accuracy gộp 0.927

StratifiedKFold 3 fold của bài trên 96 mẫu: mỗi fold, model chưa thấy các ô viền cam sẽ đoán chúng; xác suất (xanh 0, cam 1) ghi về đúng vị trí cho đủ cột OOF.

Ở TimeSeriesSplit, những mẫu đầu thường chỉ làm train, chưa bao giờ làm validation. Vì vậy không được ép OOF mọi hàng đều đầy; cần giữ mask các hàng thực sự đã được chấm và chỉ tính metric trên chúng. Không dùng các dự đoán train để lấp chỗ trống rồi gọi cả mảng là OOF.

HỎI NHANH

Một đội dùng điểm OOF để chọn feature, rồi lại dùng nó để chọn giữa nhiều mô hình. Điểm OOF của phương án thắng còn được coi là điểm test độc lập không?

A Còn, vì mỗi mẫu không nằm trong train của fold chấm nó

B Còn, nếu mọi hàng đều được chấm đúng một lần

C Không, vì các lựa chọn đều đã dựa vào điểm OOF

D Còn, nếu tăng số fold để bớt nhiễu

Ví dụ tính tay

Có 12 mẫu, ba nhãn 1 và chín nhãn 0. Chọn ngẫu nhiên ba mẫu làm validation mà không stratify. Số bộ ba có thể chọn là $\binom{12}{3} = 220$; số bộ chỉ gồm nhãn 0 là $\binom{9}{3} = 84$. Xác suất không có nhãn 1 là $84/220 \approx 0.382$. Dữ liệu mất cân bằng làm "chia ngẫu nhiên một lần" có thể rất thiếu thông tin.

Với ba fold có 10 mẫu mỗi fold và accuracy 0.7, 0.8, 0.9, số đúng là 7, 8, 9. Accuracy gộp bằng $24/30 = 0.8$, trùng trung bình fold trong trường hợp này. Nếu số mẫu khác nhau, hãy tính lại số đúng chia tổng số mẫu.

Bẫy cài đặt

Đừng reset index một bảng mà không reset hoặc căn lại bảng nhãn tương ứng. Dùng một mảng chỉ số chung cho feature, nhãn và group. Kiểm hash hoặc đặc trưng nhận diện để tìm các bản gần trùng, vì "không trùng ID" chưa chắc "không trùng nội dung".

Không fit scaler hay chọn feature trên toàn bộ dữ liệu rồi mới cross-validation. Mỗi fold phải học cả tiền xử lý từ train của chính nó; Pipeline ở bài sau giúp giữ quy tắc này. Fold

cũng không thay thế test độc lập khi đã thử nhiều phương án.

Khi đi thi

- **Hỏi trước "mẫu mới là ai"**: một hàng, một người/nhóm mới, hay một thời điểm tương lai; từ đó chọn random, group hay time split.
- **Cố định chỉ số fold** (lưu lại hoặc cố định seed) và dùng cùng fold cho mọi phương án.
- **In kiểm mỗi fold**: số mẫu, tỉ lệ lớp, giao của group, khoảng thời gian.
- **Split khó mà đúng hơn split dễ mà sai**: điểm thấp hơn nhưng dự báo được điểm thật khi nộp bài.

Tóm tắt

- Split theo câu hỏi sử dụng, không theo điểm cao nhất.
- Stratified giữ lớp; group giữ đơn vị chưa thấy; thời gian giữ hướng quá khứ–tương lai.
- OOF phải ghi vào đúng chỉ số và kiểm những hàng thực sự được chấm.
- CV giúp chọn phương án; test độc lập đánh giá sau lựa chọn.

Kiểm tra cuối bài

CÂU 1

Mô hình sẽ dùng cho những người chưa có trong dữ liệu, mà mỗi người có nhiều bản ghi. Ta nên chia dữ liệu thế nào?

A Chia theo nhóm, mỗi người là một nhóm

B Chia từng bản ghi, giữ tỉ lệ nhãn, không quan tâm người

C Chia ngẫu nhiên từng bản ghi thành K fold, vì như vậy có nhiều hàng hơn

D Dùng TimeSeriesSplit theo thứ tự dòng, dù không phải bài dự báo tương lai

CÂU 2

Hai cách làm trên cùng một bộ dữ liệu.

A: chia 70% train, 30% test. Lặp 50 vòng "train, xem điểm test, chỉnh mô hình", rồi báo điểm test cao nhất là 94.2%.

B: chia 60% train, 20% validation, 20% test. Chỉnh mô hình theo validation, rồi chấm test đúng một lần, được 91.8%.

Con số nào đáng tin hơn để ước lượng mô hình làm tốt đến đâu trên dữ liệu mới?

A Của A, vì nhiều dữ liệu train hơn và tìm kiếm kỹ hơn

B Của A, vì xem test 50 lần thì ổn định hơn một lần

C Của B, vì test của B không tham gia việc chọn

D Như nhau, vì hai tập test gần bằng nhau

ĐỀ THI NOAI 2026 Singapore · Vòng sơ loại · Câu 303

CÂU 3

Train có học sinh của trường 1 và 2. Validation có học sinh của trường 2 và 3. Không có học sinh nào ở cả hai phía. Cách chia này có kiểm được khả năng đoán cho trường mới không?

A Chưa, vì trường 2 có mặt ở cả train và validation

B Có, vì đã chia theo từng học sinh

C Có, vì các trường ở hai phía không giống hệt nhau

D Có, nếu tỉ lệ nhãn ở hai phía bằng nhau

CÂU 4

Lớp hiếm chỉ có 4 mẫu. Ta muốn chia thành 5 fold validation không chồng lên nhau, fold nào cũng có ít nhất một mẫu của lớp hiếm. Làm được không?

A Được, nếu xáo dữ liệu đủ nhiều lần

B Được, bằng cách cho một mẫu hiếm vào hai fold

C Được, vì stratified split giữ tỉ lệ lớp

D Không được, vì cần ít nhất 5 mẫu của lớp hiếm

CÂU 5

Dữ liệu dự báo có cột `time`, nhưng các hàng đang bị xáo trộn. Trước khi dùng TimeSeriesSplit, ta phải làm gì để fold nào cũng học quá khứ, đoán tương lai?

A Sắp theo y để validation có nhãn mới hơn

B Giữ thứ tự đang xáo, chỉ đổi `random_state`

C Sắp các hàng tăng dần theo `time`, rồi kiểm khoảng trống ở ranh giới

D Không cần làm gì, vì TimeSeriesSplit tự sắp theo cột `time`

CÂU 6

Hai fold validation có 10 và 30 mẫu, accuracy lần lượt là 0.6 và 0.8. Gộp cả 40 dự đoán OOF lại thì accuracy là bao nhiêu?

A 1.40

B 0.75

C 0.80

D 0.70

CÂU 7

Bài phân loại nhị phân. Mỗi fold cho ra `predict_proba(X_valid)[:, 1]`, và các fold không lấy hàng theo thứ tự. Ta ghi các dự đoán vào mảng OOF thế nào để mỗi số nằm đúng hàng của nó?

A Ghi lần lượt vào các vị trí đầu của mảng OOF

B Ghi vào đúng các vị trí `valid_idx` của mảng OOF

C Sắp các xác suất tăng dần, rồi ghép với y theo thứ tự cũ

D Nối các mảng dự đoán lại, rồi so với y theo thứ tự cũ

CÂU 8

Với TimeSeriesSplit, các hàng đầu tiên chưa bao giờ nằm trong validation. Ở những hàng ấy, mảng OOF nên ghi gì?

A Ghi nhãn thật cho đủ hàng khi chấm

B Ghi dự đoán của mô hình trên train

C Ghi 0 cho đủ shape, rồi chấm mọi hàng

D Đánh dấu là chưa chấm, chỉ tính trên các hàng có OOF

CÂU 9

Câu nào mô tả đúng K-fold cross-validation cho linear regression với một feature x ?

A Sắp theo x, cắt thành K khối liền nhau, lần lượt giữ lại một khối

B Học một lần trên toàn bộ dữ liệu, rồi chấm K lần

C Học trên mọi tập con cỡ K, lấy trung bình lỗi trên một tập test riêng

D Chia ngẫu nhiên thành K phần, mỗi vòng giữ lại một phần để chấm

PHỎNG THEO CMU 10-601 · Practice Problems, Exam 1 · Q3.4

CÂU 10

Ta dự báo lúc 8 giờ sáng thứ Hai. Feature A là tổng số đăng ký của tuần trước. Feature B là tổng số người đến trong chính tuần đang dự báo. Cả hai đều có sẵn trong bảng lịch sử, và lúc 8 giờ ta đã biết A. Được dùng feature nào?

A Chỉ A, vì lúc dự báo chưa thể biết B

B Cả A và B, vì cả hai đều có trong bảng lịch sử

C Chỉ B, vì B gần với nhãn hơn

D Không dùng A, vì dữ liệu tuần trước của các hàng có thể trùng nhau

Đọc thêm

- scikit-learn — Cross-validation. Tra sự khác nhau giữa KFold, StratifiedKFold, GroupKFold và TimeSeriesSplit.
- James và cộng sự — An Introduction to Statistical Learning, chương 5. Đọc về holdout, K-fold và sai số của phép ước lượng.

- scikit-learn — Nested versus non-nested cross-validation. Xem vì sao chọn cấu hình và đánh giá quy trình là hai việc khác nhau.